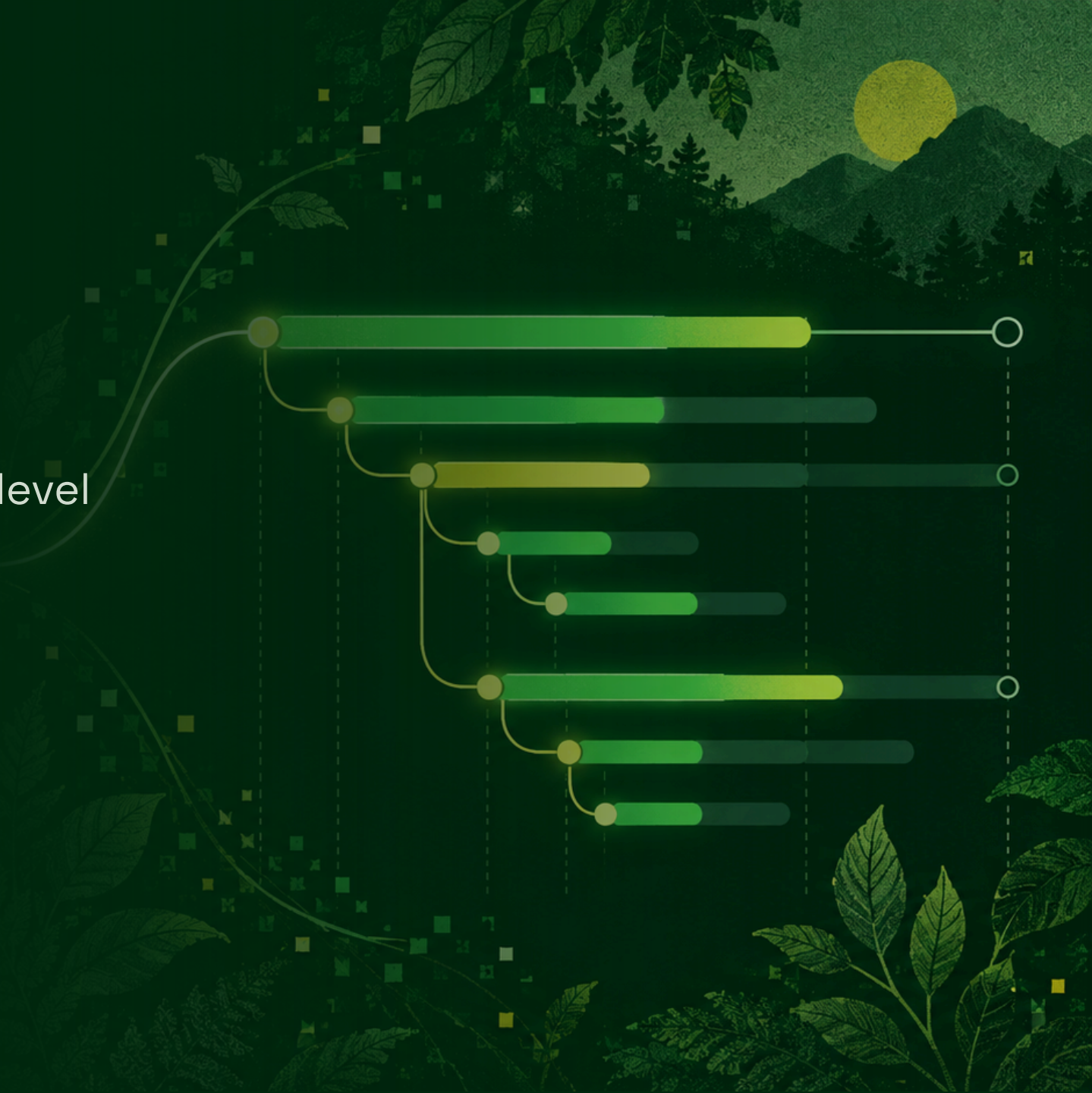


I know OTel

Giving your coding agents expert-level
OpenTelemetry knowledge



The trust test

■ ■ Where we are going

01	02	03	04	05
The hidden failure	The knowledge layer	The comparison	The demonstration	The mental model

The code worked. The telemetry lied.

The pull request compiled, passed its tests, and produced traces. It was still wrong.

■ ■ Working is not trustworthy

CI / build

PASS

go test ./... ✓

otelc build ✓

service start ✓

What the build cannot tell you

- ✗ Dependencies came from 2024
- ✗ HTTP and database spans split apart
- ✗ Span names ignored current conventions
- ✗ Sensitive values entered the pipeline

01

Models learn yesterday's OpenTelemetry

The problem starts before the prompt.



■ ■ Training cannot follow release velocity

Models freeze a moving ecosystem into their training data.

- SDK versions become stale.
- Semantic conventions evolve separately.
- New instrumentation approaches appear.

1-2

weeks between
Collector releases

2024

version chosen in
one test

v1.1.0

current otelc
target

■ ■ Four failures hide behind green builds

01	02	03	04
Stale versions	Wrong conventions	Broken context	Leaked values
Old packages compile while missing current fixes and capabilities.	Plausible attribute names break standard queries and dashboards.	Valid spans become disconnected traces at service boundaries.	Useful-looking attributes expose personal or sensitive information.

A longer prompt is not a **knowledge source.**

More instructions cannot supply facts the model does not have.

02

Give the agent a maintained knowledge layer

Current facts enter when the task needs them.



■ ■ What an Agent Skill provides

ENTRY

A small instruction surface for one domain

REFERENCES

Focused material selected for a specific task

TOOLS

Precise lookups against upstream information

WORKFLOW

Repeatable steps with explicit verification

Good instrumentation needs ■ ■ three knowledge layers

UPSTREAM

Facts

Released versions, supported configuration, semantic conventions, and current APIs.

ENGINEERING

Judgment

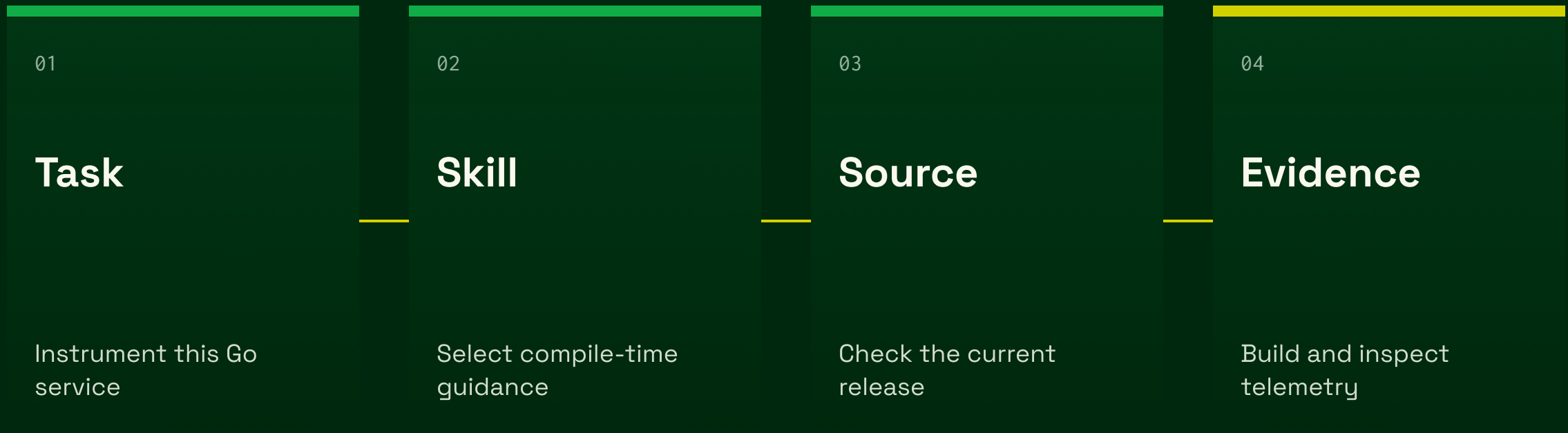
Which operations deserve spans, metrics, or logs, and which details create noise.

YOUR SYSTEM

Context

Architecture, privacy rules, service boundaries, and the questions operators ask.

■ ■ Retrieve only what the task needs



Small retrieval keeps context focused and makes the recommendation traceable.

■ ■ Fourteen skills cover the working surface

LANGUAGES

Go · Java · Node.js
Python · .NET · Browser

PIPELINE

Collector · OTTL
Telemetry generation

CORRECTNESS

SDK versions · Conventions
Breaking changes

EMERGING

Weaver · Declarative config
Span-event migration

03

Hold everything else constant

Same repository. Same model. Same task.



■ ■ One task, two knowledge conditions

THE TASK

Instrument this Go + Gin application with OpenTelemetry Go Compile-Time Instrumentation. Produce a runnable result.

BARE

Model knowledge

Web and terminal tools

SAME

repository
model
prompt

SKILLED

Model + otel-go

Web, terminal, and skill reader

■ ■ Compare outcomes, not confidence

CHECK				
			BARE	SKILLED
otelc version			v1.1.0	v1.1.0
Tests + build			PASS	PASS
HTTP trace captured			YES	YES
Database spans captured			NOT CHECKED	YES
Repeatable workflow			Makefile + docs	Tool pin + script + CI
			11:07	9:46
			Both produced valid instrumented builds.	

■ ■ Better context cost less in this run

BARE	WITH OTEL-GO	
1,910,954 total token traffic	1,160,192 total token traffic	39.3% fewer tokens 750,762 fewer

Web discovery calls fell from 8 to 3. Tool calls fell from 47 to 34.

■ ■ Verification happens outside the model

01	02	03	04
Clean build	Fixed runtime	One request	Real signals
<code>go test ./...</code>	<code>otelc build ./...</code>	<code>GET /api/articles</code>	<code>trace · metrics · logs</code>

The model does not grade its own work.

04

Now inspect what actually runs

The backend gets the final vote.



■ ■ The demonstration has three checks

01 Build with otelc The repository retains a version-pinned command.	02 Send one request HTTP and database work follow one deterministic path.	03 Inspect the result One connected trace and correlated logs reach the backend.
---	--	---

A captured run follows the same sequence if the live environment fails.

LIVE DEMO

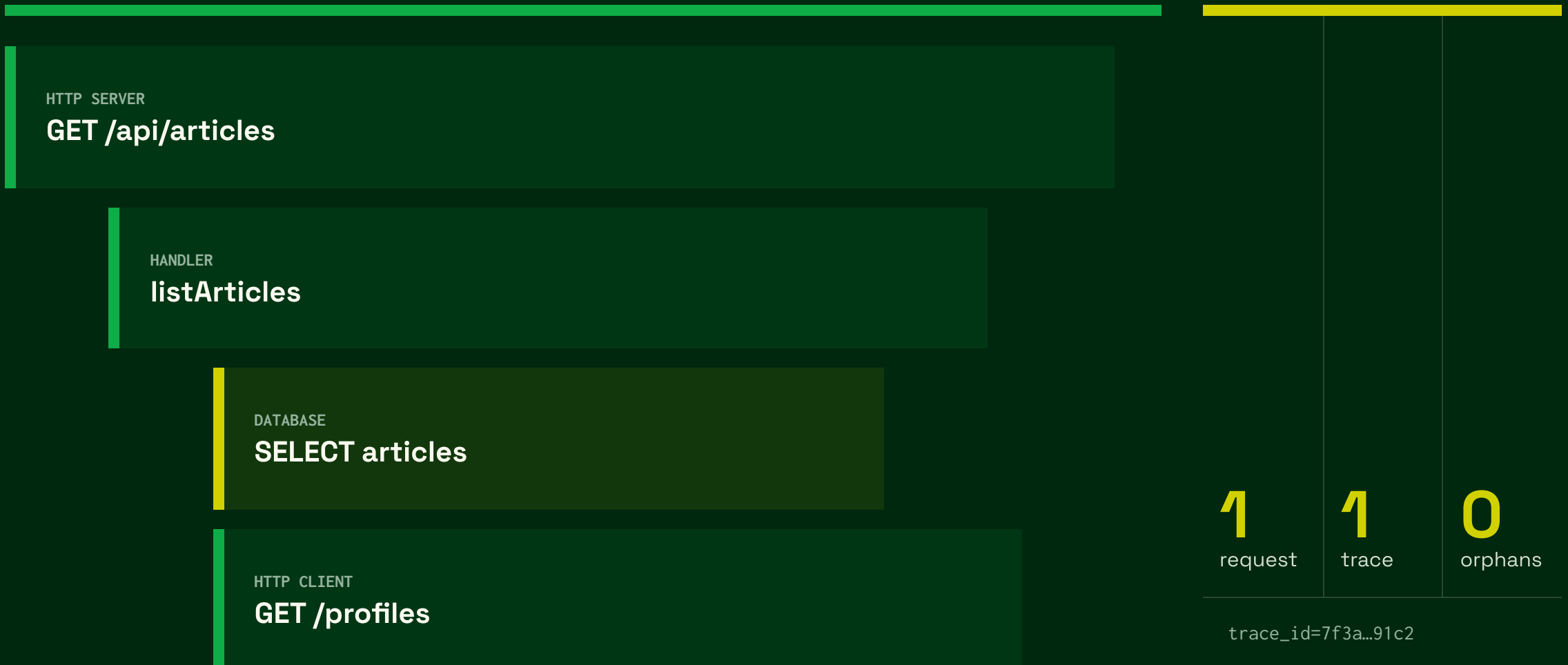
Instrument the application

One prompt. One repository. One request.

```
$ ./scripts/build-instrumented.sh  
$ ./realworld  
$ curl localhost:8080/api/articles
```

expected: 200 OK

■ ■ One request tells one connected story



■ Skills can review existing instrumentation

TELEMETRY	SOURCE	CHANGE
<pre>user.email=jo...@example.com</pre>	<pre>span.SetAttributes(attribute.String("user.email", email),)</pre>	<pre>span.SetAttributes(attribute.String("user.id", opaqueID),)</pre>
Finding: sensitive value	Trace the value to one statement	Keep the useful, non-sensitive key

Supply current facts, then **verify reality.**

Skills improve the starting point. Running telemetry decides whether the result is trustworthy.

■ ■ Try one task next week

01	02	03
Install the skills Add the open source repository to your coding agent.	Repeat one task Run the same OpenTelemetry request with and without skills.	Inspect the telemetry Judge traces, metrics, and logs in a real backend.

github.com/ollygarden/opentelemetry-agent-skills



Questions?

Juraci Paixão Kröhling

Founder and CEO, OllyGarden

`github.com/ollygarden/
opentelemetry-agent-skills`

`talks.kroehling.de`



Open source · Vendor neutral